# Lecture 10: Database Implementation

## Workshop: Revised Design and Implementation

## ITCS216 Fundamentals of Database Systems

Semester 1/2026


Asst. Prof. Dr. Preecha Tangworakitthaworn preecha.tan@mahidol.ac.th
Asst. Prof. Dr. Siripen Pongpaichet siripen.pon@mahidol.ac.th
Asst. Prof. Dr. Jidapa Kraisangka jidapa.kra@mahidol.ac.th

# Lesson Learning Outcome

After the workshop, stude are able to practice:

- Revising the Entity Relationship Diagram
- Converting 8-step transformation
- Creating a data dictionary that conform to the relational schema
- Using SQL commands to create databases and tables

# Step 1: Revise the Selected EERD

- **Format: Group Discussion (15-45 min)**
- From Project Phase I report, all members of the group **pick 8-10 entities** with the following criteria:
  - All selected entities must be connected
  - Include at least one supertype entity and their subtypes (Only 2 subtypes are recommended)
  - Must have at least one weak relationship
  - Must have at least one multi-valued attributes
  - Must have at least one M-N relationship
- **Verify to your Instructor for the selection**
- Revise your Design to meet the criteria if needed

# Step 2: Transform to Relational Model

- **Format: Individual Work from the group project (15-30 min)**
- Each member selects a *relationship* and their participated entities from the selected ERD/EERD in Step 2).
  **Note**: each member must not select the same relationship.
- Transform the selected entities and relationship into the relational model using the 8-step transformation
  **Note**: Show Step-by-step

# 8 Steps to Transform ERD to Relational Model

1. Transforming regular entity types
2. Transforming weak entities
3. Transforming binary 1:1 relationship types
4. Transforming binary 1:N relationship types
5. Transforming binary M:N relationship types
6. Transforming multi-valued attribute *A* in relation *K*
7. Transforming *n-ary* relationship types
8. Transforming specialization or generalization

***Each step can be skipped if your ERD/EERD does not contain those criteria***

# Step 3: Create a Data Dictionary

- **Format: Individual Work from the group project (15-30 min)**
- Create a data dictionary based on the relational model (only the one involved in your part in Step 2)

| Table Name | Attribute Name | Contents | Types | Format | Nullable | Range | Key | FK Referenceable Table |
|---|---|---|---|---|---|---|---|---|
| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |
| | | | | | | | | |

# RECAP: Data Dictionary

| Element | Description |
|---|---|
| **Table Name** | A relation name (usually from your relational data model) |
| **Attribute Name** | The attribute name belonged to the table |
| **Contents / Description** | Explanation of the attributes |
| **Data Type** | Defines the type of data stored (e.g., `INT`, `VARCHAR`, `DATE`) |
| **Format** | Specifies the pattern or structure for data input/output of the given attributes (e.g., `yyyy-mm-dd`). |
| **Nullable** | Indicates whether the attribute can contain `NULL` (empty) values. |
| **Range / Domain** | Acceptable range or set of valid values for the attribute. |
| **Key** | Identifies the primary key (PK) or foreign key (FK) used for data relationships. |
| **Referenced Table** | Indicates the table and column linked through a foreign key. |
| **Default Value** | Predefined value assigned if no data is entered. |

**Note:** These components are commonly included in a data dictionary, but are not limited to those listed here.

# RECAP: Data Types in RDBMS

| Data Type | Explanation | Example Usage |
|---|---|---|
| **CHAR(n)** | *Fixed-length* character string. Always stores exactly `n` characters, padding with spaces if shorter. | `CHAR(5) → 'ICT  ', CHAR(7) → '6888742'` |
| **NCHAR(n)** | *Fixed-length Unicode* character string for multilingual text (supports languages like Thai, Chinese, etc.). | `NCHAR(10) → 'มหิดล    '` |
| **VARCHAR(n)** | *Variable-length* character string. Uses only as much space as needed (up to n). | `VARCHAR(50) → 'Database Systems'` |
| **NVARCHAR(n)** | *Variable-length Unicode* string for multilingual text, supports Unicode characters. | `NVARCHAR(100) → 'เทคโนโลยีสารสนเทศ'` |
| **INT** | Integer (whole number) type for numeric data without decimals. | `INT → 12345` |
| **DECIMAL(p, s)** | Fixed-point number with `p` total digits and `s` digits after the decimal point. | `DECIMAL(10,2) → 12345.67` |
| **DATE** | Stores calendar date only (year, month, day). | `DATE → '2025-10-07'` |
| **DATETIME** | Stores both date and time (year, month, day, hour, minute, second). | `DATETIME → '2025-10-07 10:30:00'` |
| **BIT** | Boolean-like field storing 1 (true), 0 (false), or NULL. | `BIT → 1 (true)` |

# Step 4: Write SQL Scripts

- **Format: Individual Work from the group project (15-30 min)**
- Use the SQL DDL commands to create the database and the tables according to your data dictionary from Step 3
- Name your Database as:

  `[section][group ID][last 3 digits of your ID]`

  **Example** `sec4group042042`

# RECAP: SQL DDL Commands

- **Create** a database with a specified name

```
CREATE DATABASE [IF NOT EXISTS] databasename;
```

Example:

```
CREATE DATABASE IF NOT EXISTS tinycompany;
```

- **Use** a database with a specified name to set the active database

```
USE databasename; -- e.g. USE tinycompany;
```

# Example: Create a `department` Table

| Table Name | Attribute Name | Contents | Type | Format | Nullable | Range | Key | FK Referenced Table |
|---|---|---|---|---|---|---|---|---|
| Department | dnumber | Department's number | int | x | | 1 to 20 | PK | |
| | dname | Department's name | varchar(20) | Xxxxxx | | | | |
| | location | Department's main location | varchar(100) | Xxxxxx | Y | | | |

**DEPARTMENT**

| DNUMBER | DNAME | LOCATION |
|---|---|---|

```
CREATE TABLE department(
      dnumber              INT              PRIMARY KEY,
      dname                VARCHAR(20)      NOT NULL,
      location             VARCHAR(100),
);
```

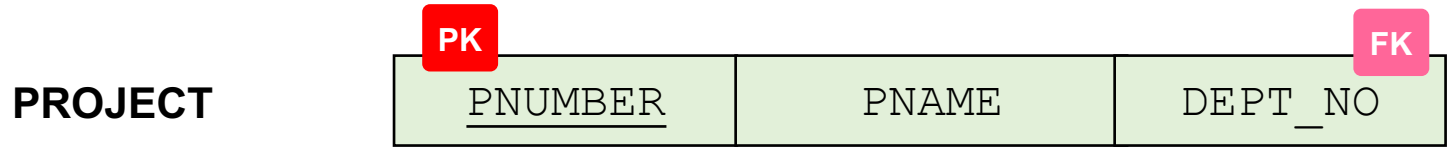
**Option 1**: Declare a primary key constraint immediately after the attribute definition. Common for **single-column primary keys**.

```
CREATE TABLE department(
      dnumber              INT,
      dname                VARCHAR(20)      NOT NULL,
      location             VARCHAR(100),
      CONSTRAINT PK_Dept PRIMARY KEY (dnumber)
);
```

**Option 2**: Declare a primary key constraint at the end of the attribute list
Require for defining a composite key

# Example: Create a `project` Table

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| Project | pnumber | Project's number | int | x | | | PK | |
| | pname | Project's name | varchar(50) | Xxxxxx | | | | |
| | dept_no | Department's number | int | x | | | FK | dnumber [Department] |




```
CREATE TABLE project(
      pnumber      INT           ,
      pname        VARCHAR(50) NOT NULL,
      dept_no      INT,
      CONSTRAINT PK_Proj PRIMARY KEY (pnumber),
      CONSTRAINT FK_DeptProj FOREIGN KEY (dept_no)
      REFERENCES department(dnumber)
);
```

Constraint name

# Submission

- Submit the selected EER/EERD, steps and the final transformed relational model, the design data dictionary, and the SQL script by the end of the class individually.
- PDF only by naming

  ITCS216-InclassActivity-Lect10-6X88XXX.pdf